

华为“智能基座”系列课程

基于昇腾 CANN 的 Atlas 200I DK A2 推理实践

版本：2.4



华为技术有限公司

版权所有 © 华为技术有限公司 2024。保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明



和其他华为商标均为华为技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受华为公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，华为公司对本文档内容不做任何明示或暗示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

华为技术有限公司

地址：深圳市龙岗区坂田华为总部办公楼 邮编：518129

网址：<http://e.huawei.com>

目录

1 实验环境介绍.....	3
1.1 实验介绍.....	3
1.1.1 关于本实验.....	3
1.1.2 实验环境.....	3
1.1.3 实验目的.....	4
1.1.4 实验清单.....	4
2 GoogLeNet 图像分类推理实践.....	5
2.1 实验介绍.....	5
2.2 实验目的.....	5
2.3 实验原理.....	5
2.4 实验流程.....	6
2.4.1 下载项目.....	6
2.4.2 离线模型转换.....	7
2.4.3 执行模型推理.....	9
2.5 关键代码解析.....	10
2.5.1 运行管理资源申请.....	11
2.5.2 加载模型文件，构建模型输出内存.....	12
2.5.3 读取本地图像数据并进行预处理.....	13
2.5.4 构建模型输入数据，进行模型推理.....	14
2.5.5 模型推理.....	14
2.5.6 解析模型推理结果.....	15
2.5.7 资源释放.....	15
2.6 实验总结.....	16
3 YOLOv3 目标检测推理实践.....	17
3.1 实验介绍.....	17
3.2 实验目的.....	17
3.3 实验流程.....	17
3.3.1 下载项目.....	17
3.3.2 离线模型转换.....	18
3.3.3 执行模型推理.....	19
3.4 实验总结.....	21
4 DeepLabv3+ 图像分割推理实践.....	21
4.1 实验介绍.....	21

4.2 实验目的.....	22
4.3 实验流程.....	22
4.3.1 下载项目.....	22
4.3.2 离线模型转换.....	23
4.3.3 执行模型推理.....	24
4.4 实验总结.....	25
5 MobileNetv2 垃圾分类训练+推理实践.....	26
5.1 实验介绍.....	26
5.1.1 实验介绍.....	26
5.1.2 数据集介绍.....	26
5.1.3 模型介绍.....	27
5.1.4 实验环境.....	28
5.1.5 实验目的.....	29
5.2 MindSpore 模型训练.....	29
5.2.1 环境准备.....	29
5.2.2 项目下载.....	29
5.2.3 导入模块.....	30
5.2.4 数据集预处理.....	30
5.2.5 模型构建.....	33
5.2.6 模型训练与验证.....	39
5.3 Atlas 200I DK A2 端侧部署.....	45
5.3.1 环境准备.....	46
5.3.2 下载项目.....	46
5.3.3 离线模型转换.....	47
5.3.4 执行模型推理.....	47
5.4 实验小结.....	48

1 实验环境介绍

1.1 实验介绍

1.1.1 关于本实验

本系列实验提供了多种场景的 AscendCL 推理应用开发样例，将其部署在新一代开发者套件 Atlas 200I DK A2 上执行推理，帮助开发者快速入门，进而熟练掌握 CANN 和 Atlas 的推理应用开发。



图1-1 Atlas 200I DK A2

1.1.2 实验环境

本实验在新一代开发者套件 Atlas 200I DK A2 上执行，因此需要部署 Atlas 200I DK A2 的实验环境，请参考《手把手教你搭建 Atlas 200I DK A2 开发环境》实验手册。



手把手教你搭建Atlas200I DK A2开发

实验环境	实验平台	AI 计算框架	AI 处理器/算力	系统/软件
Desktop-Image_23.0.RC2_ubuntu22.04	Atlas 200I DK A2 开发者套件	MindSpore2.1	Ascend 310B	Ubuntu 22.04 LTS Arm64 , npu-drive 23.0.RC2 , CANN6.2.RC2 ,

				MobaXterm 23
--	--	--	--	--------------

表1-1 实验环境

1.1.3 实验目的

通过本章实验，熟悉并掌握：

- Atlas 200I DK A2 开发者套件环境搭建；
- ATC 离线模型转换；
- AscendCL 推理应用开发；
- AI 模型训练+推理全流程开发。

1.1.4 实验清单

实验	简述	难度	实验环境/平台
GoogLeNet 图像分类推理实践	使用 GoogLeNet 模型对 ImageNet 图片进行分类推理。	初级	Atlas 200I DK A2 开发者套件
YOLOv3 目标检测推理实践	使用 YOLOv3 模型对 COCO 图片进行目标检测推理。	中级	Atlas 200I DK A2 开发者套件
DeepLabv3+图像分割推理实践	使用 DeepLabv3+模型对 COCO 图片进行图像分割推理。	中级	Atlas 200I DK A2 开发者套件
MobileNetv2 垃圾分类训练+推理实践	使用 MindSpore 构建并训练 MobileNetv2 垃圾分类模型，将其部署到 Atlas 200I DK A2 上开发推理应用。	高级	Ascend 910 算力平台（华为云 ModelArts）
			Atlas 200I DK A2 开发者套件

2 GoogLeNet 图像分类推理实践

2.1 实验介绍

本实验使用昇腾 CANN 在 Atlas 200I DK A2 上开发 GoogLeNet 推理应用，首先使用 ATC 工具对 GoogLeNet 模型进行离线模型转换，接下来使用 AscendCL 开发推理代码，实现 ImageNet 图片分类功能，输入样例是待推理的 jpg 图片，输出样例是推理后的 jpg 图片。

2.2 实验目的

- 了解如何在 Atlas 200I DK A2 上进行模型转换；
- 了解如何在 Atlas 200I DK A2 上申请运行管理资源；
- 了解如何加载模型文件并构建输出内存；
- 了解如何获取图像进行预处理；
- 了解如何在 Atlas 200I DK A2 上进行模型推理并解析推理结果。

2.3 实验原理

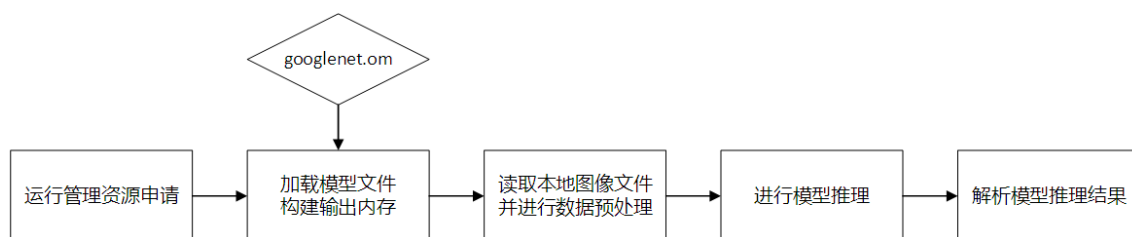


图2-1 实验流程

运行管理资源申请：用于初始化系统内部资源，固定的调用流程。

加载模型文件并构建输出内存：从文件加载离线模型数据，需要由用户自行管理模型运行的内存，根据内存中加载的模型获取模型的基本信息包含模型输入、输出数据的数据 buffer 大小；由模型的基本信息构建模型输出内存，为接下来的模型推理做好准备。

获取本地图像并进行预处理：从本地存放图像数据的目录中循环读取图像数据并使用 DVPP 的 JPEGD 功能将图片数据解码为 YUV420SP，并使用 DVPP 的 resize 功能对图像数据进行缩放至模型要求的输入图像分辨率。

模型推理和解析模型推理结果：根据构建好的模型输入数据进行模型推理，根据模型的输出，解析图片分类的结果，获取当前图像中识别出的物体类别以及对应的置信度，将分类结果标注在图片上进行展示。

2.4 实验流程

2.4.1 下载项目

步骤 1 下载项目代码

```
git clone https://gitee.com/ascend/samples.git
```

本实验项目代码位于 root/samples/python/level2_simple_inference/1_classification/googlenet_imagenet_picture

项目目录如下：

```
root/samples/python/level2_simple_inference/1_classification/googlenet_imagenet_picture
├─data
│   # 测试图片
├─model
│   # 推理模型
├─src
│   classify.py # 推理代码
│   image_net_classes.py # 数据集标签
```

步骤 2 在 model 目录下载基于 Caffe 的 GoogLeNet 模型

```
cd
$HOME/samples/python/level2_simple_inference/1_classification/googlenet_imagenet_picture/
model
wget https://obs-9be7.obs.cn-east-2.myhuaweicloud.com/003_Atc_Models/AE/ATC%20Model/
classification/googlenet.caffemodel
wget https://obs-9be7.obs.cn-east-2.myhuaweicloud.com/003_Atc_Models/AE/ATC%20Model/
classification/googlenet.prototxt
wget https://obs-9be7.obs.cn-east-2.myhuaweicloud.com/models/googlenet_imagenet_picture-
python/insert_op.cfg
```

步骤 3 在 data 目录下载测试图片

```
cd
$HOME/samples/python/level2_simple_inference/1_classification/googlenet_imagenet_picture/
data
wget https://obs-9be7.obs.cn-east-2.myhuaweicloud.com/models/googlenet_imagenet_picture-
python/rabbit.jpg
```



图2-1 测试图片样例

2.4.2 离线模型转换

对于开源框架的网络模型（如 Caffe、TensorFlow 等），不能直接在昇腾 AI 处理器上运行推理，需要先使用 ATC（Ascend Tensor Compiler）工具将开源框架的网络模型转换为适配昇腾 AI 处理器的离线模型（*.om 文件）。

ATC 架构：

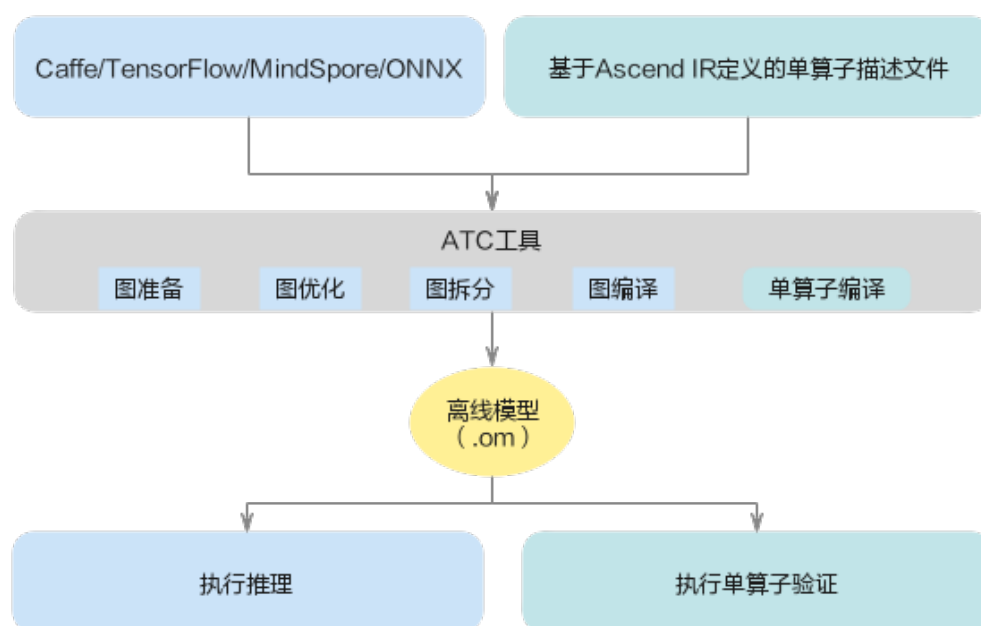


图2-1 ATC 工具功能架构

ATC 常见参数：

ATC 参数名称	参数说明
--model	原始模型文件，填写模型文件时需要带上格式，如.onnx。
--weight	原始模型权重，该参数在转换 Caffe 模型场景下使用，其他框架不使用。

--framework	原始框架类型，各框架对应的数值如下： 0:Caffe; 1:MindSpore; 3:Tensorflow; 5:ONNX。
--output	保存转换后的 om 离线推理模型文件路径。
	如果是单算子描述文件，存放转换后的单算子模型的路径。
--soc_version	昇腾 AI 处理器型号，请使用 npu-smi info 查询。
--insert_op_conf	AIPP 配置文件。
--input_shape	模型的输入节点名称和 shape，shape 的格式一般为 [batch,channels,height,width]。
--input_format	输入 Tensor 的内存排列方式，NCHW 指代 batch、channels、height、width。

表2-1 ATC 参数

步骤 1 使用 npu-smi 工具查询昇腾 AI 处理器型号

在离线模型转换前，需要使用 npu-smi 工具查询昇腾 AI 处理器型号。

npu-smi 是 NPU 的系统管理工具，可以用于收集设备信息，查看设备健康状态，对设备进行配置以及执行固件升级、清除设备信息等功能。驱动安装过程中会默认安装 npu-smi 工具。安装完成后，npu-smi 放置在“/usr/local/sbin/”路径下。

使用 npu-smi info 命令查询昇腾 AI 处理器型号：

```
npu-smi info
```

输出如下，其中昇腾 AI 处理器型号（Name）是 310B4：

```
(base) root@davinci-mini:~# npu-smi info
```

npu-smi 23.0.rc2		Version: 23.0.rc2			
NPU Chip	Name Device	Health Bus-Id	Power(W) AICore(%)	Temp(C) Memory-Usage(MB)	Hugepages-Usage(page)
0	310B4	OK	7.3	33	15 / 15
0	0	NA	0	1327 / 3513	

图2-1 npu-smi info 查询

步骤 2 使用 ATC 工具进行离线模型转换

使用 Caffe 训练的 GoogLeNet 模型在转换时需要准备模型文件（googlenet.prototxt）、权重文件（googlenet.caffemodel），以及 AIPP 配置文件 insert_op.cfg。

注意：这里的 soc_version 是使用 npu-smi info 查询到的 Ascend310B4。

```
cd
$HOME/samples/python/level2_simple_inference/1_classification/googlenet_imagenet_picture/
model
atc --model=./googlenet.prototxt --weight=./googlenet.caffemodel --framework=0 --
output=googlenet_yuv --soc_version=Ascend310B4 --insert_op_conf=./insert_op.cfg --
input_shape="data:1,3,224,224" --input_format=NCHW
```

输出“ATC Run Success”代表模型转换成功：

```
(base) root@davinci-mini:~/samples/python/level2_simple_inference/1_classification/googlenet_imagenet_picture/model# atc --model=./googlenet.prototxt --weight=./googlenet.caffe
model --framework=0 --output=googlenet_yuv --soc_version=Ascend310B4 --insert_op_conf=./
insert_op.cfg --input_shape="data:1,3,224,224" --input_format=NCHW
ATC start working now, please wait for a moment.
...
ATC run success, welcome to the next use.
```

图2-1 模型转换成功

2.4.3 执行模型推理

步骤 1 执行模型推理

classify.py 为模型推理代码，对 data 文件夹内的图片进行推理。

```
cd
$HOME/samples/python/level2_simple_inference/1_classification/googlenet_imagenet_picture/
src
python3 classify.py ../data/
```

```
(base) root@davinci-mini:~/samples/python/level2_simple_inference/1_classification/googlenet_imagenet_picture/model# cd $HOME/samples/python/level2_simple_inference/1_classification/googlenet_imagenet_picture/src
python3 classify.py ../data/
[INFO] init resource stage:
[INFO] Init resource success
[INFO] Init model resource start...
[INFO] AclLiteModel create model output dataset:
[INFO] malloc output 0, size 4000
[INFO] Create model output dataset success
[INFO] Init model resource success
resize yuv end
pre process end
post process
images:../data/rabit.jpg
===== top5 inference results: =====
label:330 confidence: 0.660645, class: wood rabbit, cottontail, cottontail rabbit
label:331 confidence: 0.337402, class: hare
label:332 confidence: 0.001743, class: Angora, Angora rabbit
label:104 confidence: 0.000003, class: wallaby, brush kangaroo
label:106 confidence: 0.000001, class: wombat
[INFO] acl resource release all resource
[INFO] dvpp resource release success
[INFO] AclLiteModel release source success
[INFO] acl resource release stream
[INFO] acl resource release context
[INFO] Reset acl device 0
[INFO] Release acl resource success
[Sample] class Samle release source success
```

图2-1 模型推理结果

步骤 2 查看推理结果

推理完成后在 out 文件夹内得到推理结果图片：

wood rabbit, cottontail, cottontail rabbit



图2-1 查看推理结果

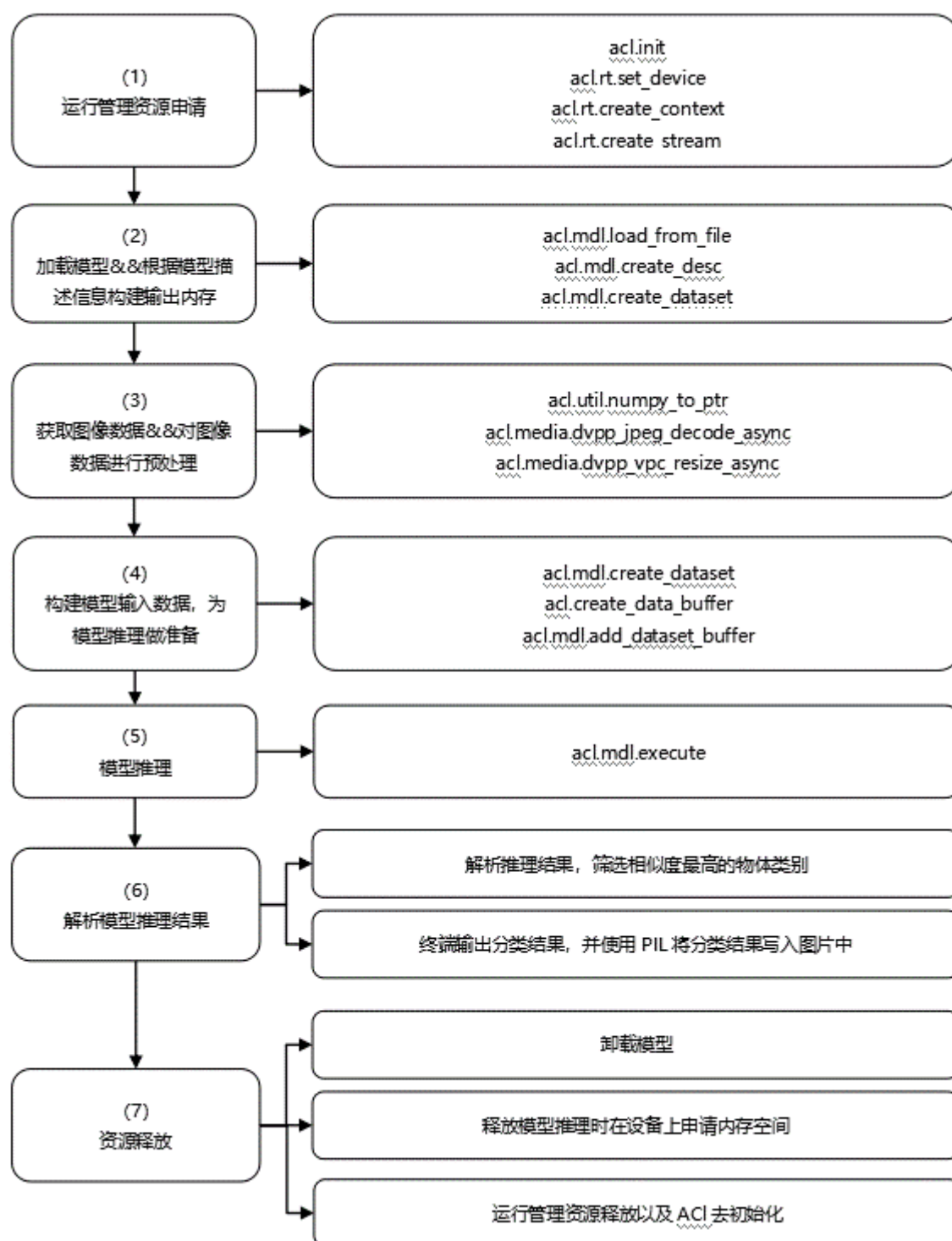
2.5 关键代码解析

ACL (Ascend Computing Language) 是一套用于在昇腾平台上开发深度神经网络推理应用的 API 库，提供运行资源管理、内存管理、模型加载与执行、算子加载与执行、媒体数据处理等 API；

ACL 利用昇腾硬件计算资源，在昇腾 CANN 平台上进行深度学习推理计算、图形图像预处理、单算子加速计算等能力。

简单来说，ACL 就是统一的 API 框架，提供 C/C++ 和 Python 编程接口，实现对所有资源的调用。

AscendCL 应用开发流程：



2.5.1 运行管理资源申请

运行管理资源申请的功能封装在了函数 `Classify._init_resource` 中。

函数源码如下所示，ACL 相关功能接口的调用已在函数中进行了说明：

```
def _init_resource(self):
    print("[Sample] init resource stage:")
    ret = acl.init()
```

```
check_ret("acl.rt.set_device", ret)

ret = acl.rt.set_device(self.device_id)
check_ret("acl.rt.set_device", ret)

self.context, ret = acl.rt.create_context(self.device_id)
check_ret("acl.rt.create_context", ret)

self.stream, ret = acl.rt.create_stream()
check_ret("acl.rt.create_stream", ret)

self.run_mode, ret = acl.rt.get_run_mode()
check_ret("acl.rt.get_run_mode", ret)

print("Init resource stage success")
```

2.5.2 加载模型文件，构建模型输出内存

加载本地 om 模型文件到内存中，并创建并获取模型的描述信息，此函数功能封装在 Model.init_resource。

函数定义及相关源码注释如下所示：

```
def init_resource(self):
    print("[Model] class Model init resource stage:")
    self.model_id, ret = acl.mdl.load_from_file(self.model_path)
    check_ret("acl.mdl.load_from_file", ret)
    self.model_desc = acl.mdl.create_desc()
    ret = acl.mdl.get_desc(self.model_desc, self.model_id)
    check_ret("acl.mdl.get_desc", ret)
    output_size = acl.mdl.get_num_outputs(self.model_desc)
    self._gen_output_dataset(output_size)
    print("[Model] class Model init resource stage success")
    self._get_output_info(output_size)

    return SUCCESS
```

根据模型的描述信息，获取模型的每路输出数据在设备上所需的空间大小。

此函数功能封装在 Model._gen_output_dataset 中。

ACL 库内置数据类型说明：aclmdlDataset 主要用于描述模型推理时的输入数据或输出数据，模型可能存在多个输入、多个输出，每个输入或输出的内存地址、内存大小用 aclDataBuffer 类型的数据来描述。

函数定义及相关源码注释如下所示：

```
def _gen_output_dataset(self, size):
    print("[Model] create model output dataset:")
    dataset = acl.mdl.create_dataset()
    for i in range(size):
        temp_buffer_size = acl.mdl.\
            get_output_size_by_index(self.model_desc, i)
```

```
temp_buffer, ret = acl.rt.malloc(temp_buffer_size,
                                ACL_MEM_MALLOC_NORMAL_ONLY)
check_ret("acl.rt.malloc", ret)
dataset_buffer = acl.create_data_buffer(temp_buffer,
                                       temp_buffer_size)
_, ret = acl.mdl.add_dataset_buffer(dataset, dataset_buffer)
if ret:
    acl.destroy_data_buffer(dataset)
    check_ret("acl.destroy_data_buffer", ret)
self.output_dataset = dataset
print("[Model] create model output dataset success")
```

2.5.3 读取本地图像数据并进行预处理

读取本地图像数据后，使用 DVPP 的 JPEGD 功能将图片数据解码为 YUV420SP 并使用 DVPP 的 resize 功能对图像数据进行缩放至模型要求的输入图像分辨率；将读取到的图像数据拷贝至设备侧申请的内存空间中，为接下来构建模型输入数据做好准备。

DVPP 的解码功能函数封装在 Dvpp.jpegd 中。

函数定义及相关源码注释如下所示：

```
def jpegd(self, image):
    print('[Dvpp] vpc decode stage:')

    device_image = image.copy_to_device(self._run_mode)
    output_desc, out_buffer = self._gen_jpegd_out_pic_desc(image)
    ret = acl.media.dvpp_jpeg_decode_async(self._dvpp_channel_desc,
                                          device_image.data(),
                                          image.size,
                                          output_desc,
                                          self._stream)

    if ret != ACL_ERROR_NONE:
        print("dvpp_jpeg_decode_async failed ret={}".format(ret))
        return None

    ret = acl.rt.synchronize_stream(self._stream)
    if ret != ACL_ERROR_NONE:
        print("dvpp_jpeg_decode_async failed ret={}".format(ret))
        return None

    width = align_up16(image.width)
    height = align_up2(image.height)
    return AclImage(out_buffer, width, height, yuv420sp_size(width, height))
```

DVPP 的大小缩放功能函数封装在 Dvpp.resize 中。

函数定义及相关源码注释如下所示：

```
def resize(self, image, resize_width, resize_height):
    print('[Dvpp] vpc resize stage:')
    input_desc = self._gen_input_pic_desc(image)
```



```

output_desc, out_buffer, out_buffer_size = \
    self._gen_resize_out_pic_desc(resize_width, resize_height)
ret = acl.media.dvpp_vpc_resize_async(self._dvpp_channel_desc,
                                     input_desc,
                                     output_desc,
                                     self._resize_config,
                                     self._stream)

check_ret("acl.media.dvpp_vpc_resize_async", ret)
ret = acl.rt.synchronize_stream(self._stream)
check_ret("acl.rt.synchronize_stream", ret)
print('[Dvpp] vpc resize stage success')

stride_width = align_up16(image.width)
stride_height = align_up2(image.height)
return AclImage(out_buffer, stride_width,
                stride_height, out_buffer_size)

```

2.5.4 构建模型输入数据，进行模型推理

构建模型的输入数据，本样例中的 googlenet 模型有一路输入，此输入是描述设备侧包含有图像数据的内存空间。

构建模型输入数据的功能函数封装在 Model._gen_input_dataset 中。

函数定义及相关源码注释如下所示：

```

def _gen_input_dataset(self, data, data_size):
    self.input_dataset = acl.mdl.create_dataset()
    input_dataset_buffer = acl.create_data_buffer(data, data_size)
    _, ret = acl.mdl.add_dataset_buffer(
        self.input_dataset,
        input_dataset_buffer)
    if ret:
        ret = acl.destroy_data_buffer(self.input_dataset)
        check_ret("acl.destroy_data_buffer", ret)

```

2.5.5 模型推理

根据已经加载到内存中，要进行推理的模型 ID、构建好的模型推理输入数据，调用 ACL 库中模型推理接口进行模型推理。

模型推理功能函数封装在了 Model.execute 中。

函数定义及相关源码注释如下所示：

```

def execute(self, data, data_size):
    self._gen_input_dataset(data, data_size)
    ret = acl.mdl.execute(self.model_id,
                          self.input_dataset,
                          self.output_dataset)
    check_ret("acl.mdl.execute", ret)
    return self._output_dataset_to_numpy()

```

2.5.6 解析模型推理结果

模型的输出为 1000 个 float 型数据，1000 个 float 型数据代表 1000 中物体类别，每个 float 型数据为归一化数据范围 0 ~ 1。代表该类物体识别的置信度，选取其中置信度最高的类别，作为图像的分类结果。

模型推理结果解析的功能函数封装在 Classify.post_process 中。

函数定义及相关源码注释如下所示：

```
def post_process(self, infer_output, image_file):
    print("post process")
    data = infer_output[0]
    vals = data.flatten()
    top_k = vals.argsort()[-1:-6:-1]
    print("images:{ }".format(image_file))
    print("===== top5 inference results: =====")
    for n in top_k:
        object_class = get_image_net_class(n)
        print("label:%d confidence: %f, class: %s" % (n, vals[n], object_class))

    #使用 pillow，将置信度最高的类别写在图片上，并保存到本地
    if len(top_k):
        object_class = get_image_net_class(top_k[0])
        output_path = os.path.join("./outputs", os.path.basename(image_file))
        origin_img = Image.open(image_file)
        draw = ImageDraw.Draw(origin_img)
        font = ImageFont.truetype("SourceHanSansCN-Normal.ttf", size=30)
        draw.text((10, 50), object_class, font=font, fill=255)
        origin_img.save(output_path)
```

2.5.7 资源释放

推理接收后要卸载模型，并释放与模型推理相关的资源，此功能函数封装在 Model.__del__ 中。

函数定义及相关源码注释如下所示：

```
def __del__(self):
    self._release_dataset()
    if self.model_id:
        ret = acl.mdl.unload(self.model_id)
        check_ret("acl.mdl.unload", ret)
    if self.model_desc:
        ret = acl.mdl.destroy_desc(self.model_desc)
        check_ret("acl.mdl.destroy_desc", ret)
    print("Model release source success")
```

释放存放图像数据的内存空间的功能函数封装在 AclImage.destroy 中。

函数定义及相关源码注释如下所示：

```
# 释放存放图像数据的内存空间
```

```
def destroy(self):
    if self._memory_type == MEMORY_DEVICE:
        acl.rt.free(self._data)
    elif self._memory_type == MEMORY_HOST:
        acl.rt.free_host(self._data)
    elif self._memory_type == MEMORY_DVPP:
        acl.rt.free_dvpp(self._data)
```

运行管理资源释放以及 ACL 去初始化的功能函数封装在 `Classify.__del__` 中。

函数定义及相关源码注释如下所示：

```
def __del__(self):
    if self._model:
        del self._model
    if self._dvpp:
        del self._dvpp
    if self.stream:
        acl.rt.destroy_stream(self.stream)
    if self.context:
        acl.rt.destroy_context(self.context)
    acl.rt.reset_device(self.device_id)
    acl.finalize()
    print("[Sample] class Sample release source success")
```

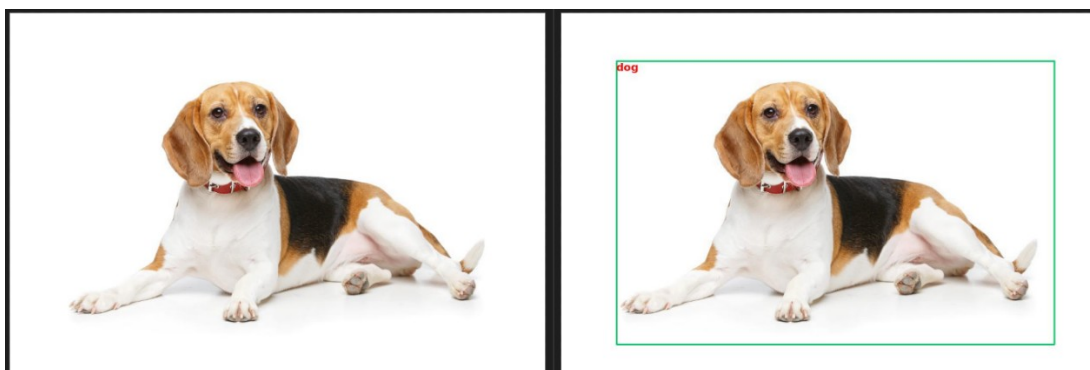
2.6 实验总结

本实验主要介绍如何使用昇腾 CANN 和 Atlas 200I DK A2 开发 GoogLeNet 的图像分类推理应用，从而熟悉并掌握 ATC 模型转换工具和 AscendCL 开发语言。

3 YOLOv3 目标检测推理实践

3.1 实验介绍

本实验使用昇腾 CANN 在 Atlas 200I DK A2 上开发 YOLOv3 目标检测项目，首先使用 ATC 工具对 YOLOv3 模型进行离线模型转换，接下来使用 AscendCL 开发推理代码，对 COCO 图片进行目标检测推理，得到检测出的物体置信度以及检测框的坐标值，最后使用 Pillow 模块将检测结果标注在原始图像上并写到本地文件中。本实验输入样例是待推理的 jpg 图片，输出样例是推理后的 jpg 图片。



3.2 实验目的

- 掌握 ATC 模型转换工具的基本操作；
- 掌握 AscendCL 开发目标检测的推理应用；

3.3 实验流程

3.3.1 下载项目

步骤 1 下载项目代码

```
git clone https://gitee.com/ascend/samples.git # 此前如已经下载，则不再执行此下载
```

本实验项目代码位于 root/samples/python/level2_simple_inference/
2_object_detection/YOLOV3_coco_detection_picture

项目目录如下：

```
root/samples/python/level2_simple_inference/2_object_detection/  
YOLOV3_coco_detection_picture
```

```
└─data
  |   # 测试图片
└─model
  |   # 推理模型
└─src
  |   object_detect.py # 推理代码
```

步骤 2 在 model 目录下载基于 Caffe 的 YOLOv3 模型

```
cd ${HOME}/samples/python/level2_simple_inference/2_object_detection/
YOLOV3_coco_detection_picture/model
wget https://obs-9be7.obs.cn-east-2.myhuaweicloud.com/003_Atc_Models/AE/ATC%20Model/
Yolov3/yolov3.caffemodel
wget https://obs-9be7.obs.cn-east-2.myhuaweicloud.com/003_Atc_Models/AE/ATC%20Model/
Yolov3/yolov3.prototxt
wget https://obs-9be7.obs.cn-east-2.myhuaweicloud.com/003_Atc_Models/AE/ATC%20Model/
Yolov3/aipp_nv12.cfg
```

步骤 3 在 data 目录下载测试图片

```
cd
${HOME}/samples/python/level2_simple_inference/2_object_detection/YOLOV3_coco_detection_p
icture/data
wget
https://obs-9be7.obs.cn-east-2.myhuaweicloud.com/models/YOLOV3_coco_detection_picture/
dog1_1024_683.jpg
```



图3-1 测试图片样例

3.3.2 离线模型转换

步骤 1 使用 ATC 工具进行离线模型转换

使用 Caffe 训练的 YOLOV3 模型在转换时需要准备模型文件 (yolov3.prototxt)、权重文件 (yolov3.caffemodel)，以及 AIPP 配置文件 aipp_nv12.cfg。

注意：这里的 soc_version 是使用 npu-smi info 查询到的 Ascend310B4。

```
cd ${HOME}/samples/python/level2_simple_inference/2_object_detection/YOLOV3_coco_detection_picture/model
atc --model=yolov3.prototxt --weight=yolov3.caffemodel --framework=0 --output=yolov3_yuv
--soc_version=Ascend310B4 --insert_op_conf=aipp_nv12.cfg
```

输出“ATC Run Success”代表模型转换成功：

```
(base) root@davinci-mini:~/samples/python/level2_simple_inference/2_object_detection/YOLOV3_coco_detection_picture/data# cd ${HOME}/samples/python/level2_simple_inference/2_object_detection/YOLOV3_coco_detection_picture/model
atc --model=yolov3.prototxt --weight=yolov3.caffemodel --framework=0 --output=yolov3_yuv
--soc_version=Ascend310B4 --insert_op_conf=aipp_nv12.cfg
ATC start working now, please wait for a moment.
...
ATC run success, welcome to the next use.
```

图3-1 模型转换成功

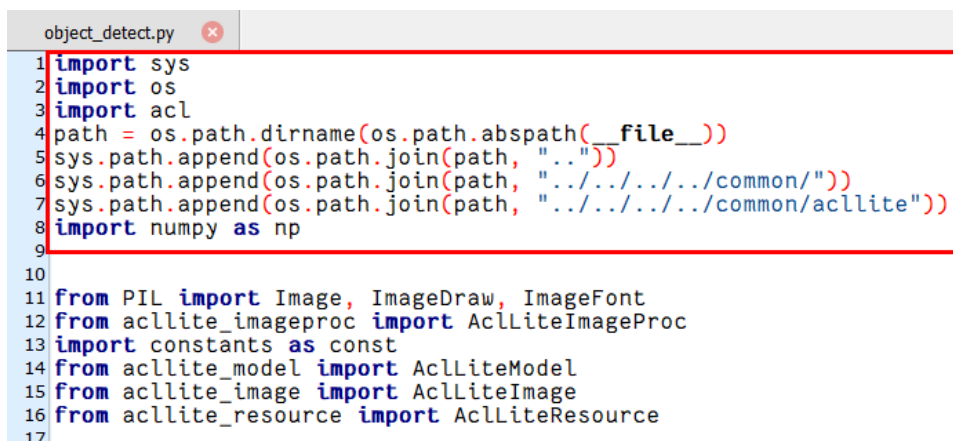
3.3.3 执行模型推理

步骤 1 修改 object_detect.py

打开 object_detect.py，在第一行插入下方代码：

```
import sys
import os
import acl
path = os.path.dirname(os.path.abspath(__file__))
sys.path.append(os.path.join(path, ".."))
sys.path.append(os.path.join(path, ".././.././common/"))
sys.path.append(os.path.join(path, ".././.././common/acllite"))
import numpy as np
```

修改完成后记得保存，如图所示：



```
object_detect.py
1 import sys
2 import os
3 import acl
4 path = os.path.dirname(os.path.abspath(__file__))
5 sys.path.append(os.path.join(path, ".."))
6 sys.path.append(os.path.join(path, ".././.././common/"))
7 sys.path.append(os.path.join(path, ".././.././common/acllite"))
8 import numpy as np
9
10
11 from PIL import Image, ImageDraw, ImageFont
12 from acllite_imageproc import AclLiteImageProc
13 import constants as const
14 from acllite_model import AclLiteModel
15 from acllite_image import AclLiteImage
16 from acllite_resource import AclLiteResource
17
```

图3-1 插入代码示例

步骤 2 执行模型推理

object_detect.py 为模型推理代码，对 data 文件夹内的图片进行推理。

```
cd ${HOME}/samples/python/level2_simple_inference/2_object_detection/YOLOV3_coco_detection_picture/src
```

```
python3 object_detect.py ../data/

(base) root@davinci-mini:~/samples/python/level2_simple_inference/2_object_detection/YOLOV3_coco_detection_picture/src# python3 object_detect.py ../data/
[INFO] init resource stage:
[INFO] Init resource success
[INFO] Init model resource start...
[INFO] AclLiteModel create model output dataset:
[INFO] malloc output 0, size 24576
[INFO] malloc output 1, size 32
[INFO] Create model output dataset success
[INFO] Init model resource success
decode jpeg end
[INFO] AclLiteImageProc vpc crop and paste stage:
[INFO] AclLiteImageProc vpc crop and paste stage success
resize yuv end
pre process end
post process
[[1 0 0 0 0 0 0]]
box num 1

[[ 55.25 ]
 [ 60.125 ]
 [ 375. ]
 [ 235.875 ]
 [ 0.984375 ]
 [ 16. ] ]
images:../data/dog1_1024_683.jpg
===== inference results: =====
dog: class 16, box 136 148 923 580, score 0.984375
[INFO] acl resource release all resource
[INFO] AclLiteModel release source success
[INFO] dvpp resource release success
[INFO] acl resource release stream
[INFO] acl resource release context
[INFO] Reset acl device 0
[INFO] Release acl resource success
```

图3-1 模型推理结果

步骤 3 查看推理结果

推理完成后在 out 文件夹内得到推理结果图片：



图3-1 查看推理结果

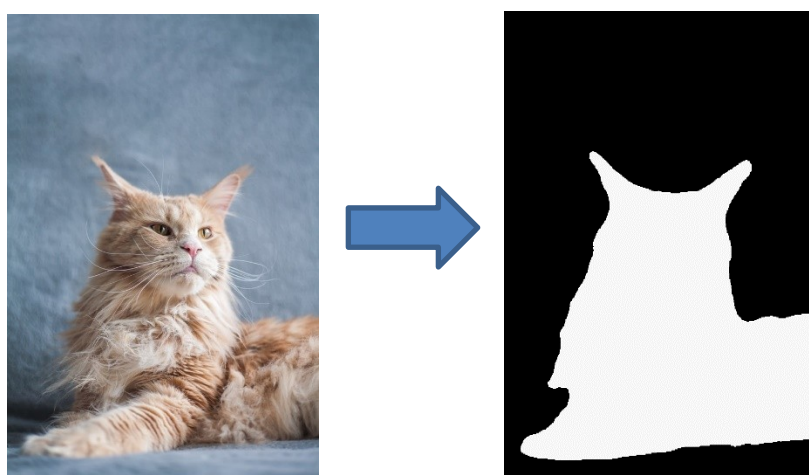
3.4 实验总结

本实验主要介绍如何使用昇腾 CANN 和 Atlas 200I DK A2 开发 YOLOv3 的目标检测推理应用，使学员进一步熟悉昇腾应用开发流程，加深对昇腾相关理论的理解。

4 DeepLabv3+图像分割推理实践

4.1 实验介绍

本实验使用昇腾 CANN 在 Atlas 200I DK A2 上开发 DeepLabV3_Plus 推理应用，首先使用 ATC 工具对 DeepLabV3_Plus 模型进行离线模型转换，接下来使用 AscendCL 开发推理代码，对图像中的物体进行语义分割，得到分割后的结果，最后使用 OpenCV 写到本地文件中。输入样例是待推理的 jpg 图片，输出样例是推理后的 jpg 图片。



4.2 实验目的

- 掌握 ATC 模型转换工具的基本操作；
 - 掌握 AscendCL 开发图像分割的推理应用；
-

4.3 实验流程

4.3.1 下载项目

步骤 1 下载项目代码

```
git clone https://gitee.com/ascend/samples.git
```

本实验项目代码位于 root/samples/python/level2_simple_inference/
3_segmentation/deeplabv3_pascal_pic

项目目录如下：

```
root/samples/python/level2_simple_inference/3_segmentation/deeplabv3_pascal_pic
├─data # 需创建
│   # 测试图片
├─model
│   # 推理模型
├─src
│   deeplabv3.py # 推理代码
```

步骤 2 在 model 目录下载基于 TensorFlow 的 DeepLapV3_Plus 模型

```
cd
${HOME}/samples/python/level2_simple_inference/3_segmentation/deeplabv3_pascal_pic/
model
wget https://obs-9be7.obs.cn-east-2.myhuaweicloud.com:443/003_Atc_Models/nkxiaolei/
DeepLapV3_Plus/deeplabv3_plus.pb
```

步骤 3 创建 data 文件夹

```
mkdir
${HOME}/samples/python/level2_simple_inference/3_segmentation/deeplabv3_pascal_pic/data
```

步骤 4 在 data 目录下载测试图片

```
cd ${HOME}/samples/python/level2_simple_inference/3_segmentation/deeplabv3_pascal_pic/
data
wget https://obs-9be7.obs.cn-east-2.myhuaweicloud.com/models/deeplabv3/girl.jpg
wget https://obs-9be7.obs.cn-east-2.myhuaweicloud.com/models/deeplabv3/dog.jpg
wget https://obs-9be7.obs.cn-east-2.myhuaweicloud.com/models/deeplabv3/cat.jpg
```



图4-1 测试图片样例

4.3.2 离线模型转换

步骤 1 使用 ATC 工具进行离线模型转换

使用 TensorFlow 训练的 DeepLabV3_Plus 模型在转换时需要准备模型文件 (deeplabv3_plus.pb)。

注意：这里的 soc_version 是使用 npu-smi info 查询到的 Ascend310B4。

```
cd
${HOME}/samples/python/level2_simple_inference/3_segmentation/deeplabv3_pascal_pic/
model
atc --model=./deeplabv3_plus.pb --framework=3 --output=deeplabv3_plus --
soc_version=Ascend310B4 --input_shape="ImageTensor:1,513,513,3" --
output_type="SemanticPredictions:0:FP32" --out_nodes="SemanticPredictions:0"
```

输出“ATC Run Success”代表模型转换成功：

```
(base) root@davinci-mini:~/samples/python/level2_simple_inference/3_segmentation/deeplab
v3_pascal_pic/src# cd ${HOME}/samples/python/level2_simple_inference/3_segmentation/deep
labv3_pascal_pic/model
atc --model=./deeplabv3_plus.pb --framework=3 --output=deeplabv3_plus --soc_version=Asce
nd310B4 --input_shape="ImageTensor:1,513,513,3" --output_type="SemanticPredictions:0:FP
32" --out_nodes="SemanticPredictions:0"
ATC start working now, please wait for a moment.
...
ATC run success, welcome to the next use.
```

图4-1 模型转换成功

4.3.3 执行模型推理

步骤 1 执行模型推理

deeplabv3.py 为模型推理代码，对 data 文件夹内的图片进行推理。

```
cd ${HOME}/samples/python/level2_simple_inference/3_segmentation/deeplabv3_pascal_pic/src
python3 deeplabv3.py ../data
```

```
(base) root@davinci-mini:~/samples/python/level2_simple_inference/3_segmentation/deeplabv3_pascal_pic/model# cd ${HOME}/samples/python/level2_simple_inference/3_segmentation/deeplabv3_pascal_pic/src
python3 deeplabv3.py ../data
[INFO] init resource stage:
[INFO] Init resource success
[INFO] Init model resource start...
[INFO] AclLiteModel create model output dataset:
[INFO] malloc output 0, size 1052676
[INFO] Create model output dataset success
[INFO] Init model resource success
/root/samples/python/level2_simple_inference/3_segmentation/deeplabv3_pascal_pic/src/../out/cat.jpg
/root/samples/python/level2_simple_inference/3_segmentation/deeplabv3_pascal_pic/src/../out/girl.jpg
/root/samples/python/level2_simple_inference/3_segmentation/deeplabv3_pascal_pic/src/../out/dog.jpg
Execute end
[INFO] acl resource release all resource
[INFO] AclLiteModel release source success
[INFO] dvpp resource release success
[INFO] acl resource release stream
[INFO] acl resource release context
[INFO] Reset acl device 0
[INFO] Release acl resource success
```

图4-1 模型推理结果

步骤 2 查看推理结果

推理完成后在 out 文件夹内得到推理结果图片：



图4-1 推理结果图片

4.4 实验总结

本实验主要介绍如何使用昇腾 CANN 和 Atlas 200I DK A2 开发 DeepLapV3_Plus 的图像分割推理应用，使学员进一步熟悉昇腾应用开发流程，加深对昇腾相关理论的理解。

5 MobileNetv2 垃圾分类训练+推理实践

5.1 实验介绍

5.1.1 实验介绍

垃圾分类有利于资源节约与循环利用，也有利于保护自然生态环境。但在垃圾分类实施过程中，市民往往并不清楚垃圾该如何分类，而且每个城市实施的垃圾分类标准也不一样，所以开发一个垃圾分类的应用有助于普及垃圾分类的知识，精确分类垃圾，解决市民的困惑，促进垃圾分类政策的执行。

我们使用现实生活中常见的垃圾分类数据集，基于 Ascend910 和 Ascend310B 算力平台，实现垃圾分类模型训练+推理部署的全流程实验。首先在 Ascend910 算力平台使用迁移学习训练得到 MobileNetv2 的垃圾分类模型，接下来将其部署在 Atlas 200I DK A2 上，使用 Ascend310B 算力平台实现端侧推理。

5.1.2 数据集介绍

在本实验中，我们将常见生活垃圾分为以下 4 大类，26 个子类：

- 干垃圾：贝壳、打火机、旧镜子、扫把、陶瓷碗、牙刷、一次性筷子、脏污衣服。
- 可回收物：报纸、玻璃制品、篮球、塑料瓶、硬纸板、玻璃瓶、金属制品、帽子、易拉罐、纸张。
- 湿垃圾：菜叶、橙皮、蛋壳、香蕉皮。
- 有害垃圾：电池、药片胶囊、荧光灯、油漆桶。

其对应的英文标签为：

- Seashell, Lighter, Old Mirror, Broom, Ceramic Bowl, Toothbrush, Disposable Chopsticks, Dirty Cloth.
- Newspaper, Glassware, Basketball, Plastic Bottle, Cardboard, Glass Bottle, Metalware, Hats, Cans, Paper.
- Vegetable Leaf, Orange Peel, Eggshell, Banana Peel.
- Battery, Tablet capsules, Fluorescent lamp, Paint bucket.

数据集目录：

```
data_en:
├─train:
│   ├─Banana Peel
│   ├─Basketball
│   ├─Battery
│   ...
└─Vegetable Leaf
```

```

└─test:
  └─Banana Peel
  └─Basketball
  └─Battery
  ...
  └─Vegetable Leaf

```

5.1.3 模型介绍

本实验我们使用 MobileNetV2 的预训练模型（即 mobilenetV2-200_1067.ckpt），对模型进行微调（Fine-tuning），只训练最后的 FC 层，并在训练过程中保存 Checkpoint。

MobileNet 网络于 2017 年提出，专注于移动端或者嵌入式设备中的轻量级 CNN 网络。相比传统卷积神经网络，MobileNet 网络使用深度可分离卷积（Depthwise Separable Convolution）的思想在准确率小幅度降低的前提下，大大减小了模型参数与运算量。并引入宽度系数 α 和分辨率系数 β 使模型满足不同应用场景的需求。

- 深度可分离卷积（Depthwise Separable Convolution）

将 Standard Convolutions（标准卷积）分解为 Depthwise Convolution（深度卷积）和 Pointwise Convolution（逐点卷积），它默认一种假设，使用分解后的卷积效果和标准卷积效果是近似的。

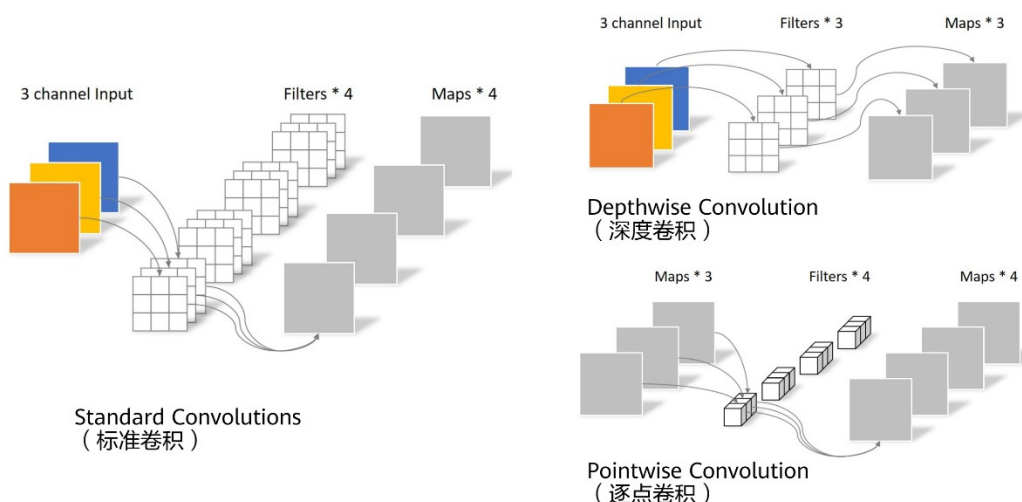


图5-1 深度可分离卷积

也就是将跨通道的 3X3 卷积换成单通道的 3X3 卷积+跨通道的 1X1 卷积：

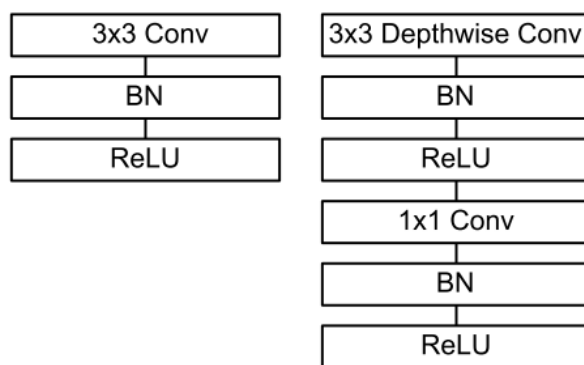


Figure 3. Left: Standard convolutional layer with batchnorm and ReLU. Right: Depthwise Separable convolutions with Depthwise and Pointwise layers followed by batchnorm and ReLU.

图5-2 深度可分离卷积

MobileNetv2 相比 MobileNetv1，有以下改进：

- 引入倒残差结构
- 采用线性瓶颈模块
- 加入了残差连接

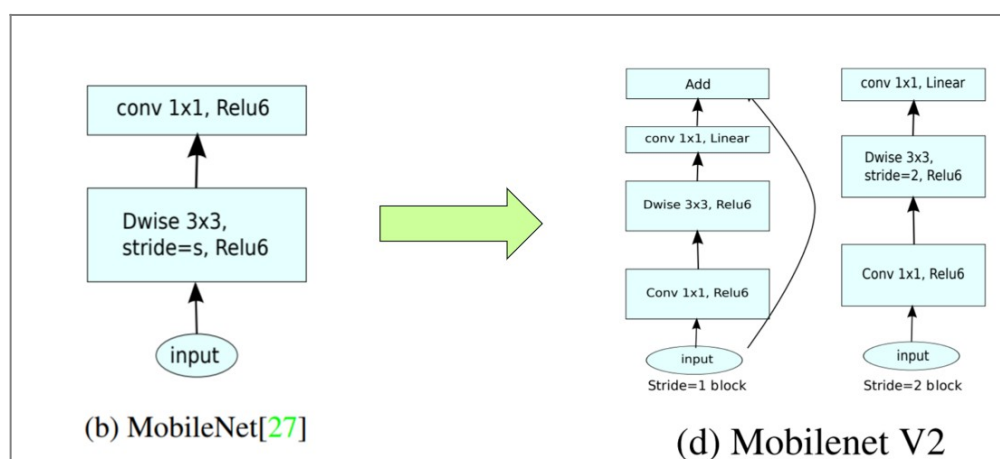


图5-3 MobileNetv2 与 MobileNetv1 的对比

5.1.4 实验环境

本实验分为两部分：

- MindSpore 模型训练：在（华为云 ModelArts 的）Ascend910 开发环境中执行。
- Atlas 200I DK A2 端侧部署：在 Atlas 200I DK A2 上的 Ascend310B 推理环境中执行。

实验环境	实验平台	AI 计算框架	AI 处理器/算力	软件
------	------	---------	-----------	----

MindSpore 模型训练	华为云 ModelArts	MindSpore2. 1	Ascend 910	Notebook 环境， Python3.7.5， MindSpore2.1
Atlas 200I DK A2 端侧 部署	Atlas 200I DK A2	无	Ascend 310B	Ubuntu 22.04 LTS Arm64， npu-drive 23.0.RC2， CANN6.2.RC2， MobaXterm 23

表5-1 环境要求

MindSpore 模型训练部分请参考官网：

https://gitee.com/ascend/samples/wikis/MobileNetV2%E5%9E%83%E5%9C%BE%E5%88%86%E7%B1%BB?sort_id=3404387

Atlas 200I DK A2 端侧部署请参考官网：

https://gitee.com/ascend/samples/tree/master/python/contrib/garbage_picture

5.1.5 实验目的

- 熟悉基于 MindSpore 的 MobileNetv2 网络构建；
- 掌握 ATC 模型转换工具的基本操作；
- 掌握 AscendCL 开发垃圾分类的推理应用；
- 掌握基于 Ascend 910 和 Ascend310B 算力的训练+推理全流程实践。

5.2 MindSpore 模型训练

5.2.1 环境准备

请参考下方《MindSpore 实验环境搭建手册》：



华为云ModelArts
环境搭建手册 (MindSpore)

5.2.2 项目下载

下载实验代码及数据集的压缩包：

https://ascend-professional-construction-dataset.obs.cn-north-4.myhuaweicloud.com:443/20230927/mobilenetv2_garbage.zip

项目目录如下：

```
mobilenetv2_garbage:
├─mobilenetv2_garbage.ipynb  # 模型训练与验证脚本
├─ckpt      # 模型保存路径
└─mobilenetV2-200_1067.ckpt  # 预训练模型
```



```
└─data_en          # 数据集
  └─test           # 测试集
    └─Banana Peel
    └─Basketball
    ...
    └─Vegetable Leaf
  └─train          # 训练集
    └─Banana Peel
    └─Basketball
    ...
    └─Vegetable Leaf
```

5.2.3 导入模块

```
import math
import numpy as np
import os
import random

from matplotlib import pyplot as plt
from easydict import EasyDict
from PIL import Image
import numpy as np
import mindspore.nn as nn
from mindspore import ops as P
from mindspore.ops import add
from mindspore import Tensor
import mindspore.common.dtype as mstype
import mindspore.dataset as de
import mindspore.dataset.vision as C
import mindspore.dataset.transforms as C2
import mindspore as ms
from mindspore import set_context, nn, Tensor, load_checkpoint, save_checkpoint, export
from mindspore.train import Model
from mindspore.train import Callback, LossMonitor, ModelCheckpoint, CheckpointConfig

os.environ['GLOG_v'] = '3' # Log level includes 3(ERROR), 2(WARNING), 1(INFO), 0(DEBUG).
set_context(mode=ms.GRAPH_MODE, device_target="Ascend", device_id=0) # 设置采用图模式
执行，设备为 Ascend#
```

5.2.4 数据集预处理

利用 ImageFolderDataset 方法读取垃圾分类数据集，并对数据集进行预处理。

读取数据集时指定训练集和测试集，首先对整个数据集进行归一化、修改图像频道等预处理操作；然后对训练集的数据依次进行

RandomCropDecodeResize、RandomHorizontalFlip、RandomColorAdjust、shuffle 操作，以增加训练数据的丰富度；接着对测试集进行 Decode、Resize、CenterCrop 等预处理操作；最后返回处理后的数据集。

```
def create_dataset(dataset_path, config, training=True, buffer_size=1000):
    """
    create a train or eval dataset

    Args:
        dataset_path(string): the path of dataset.
        config(struct): the config of train and eval in different platform.

    Returns:
        train_dataset, val_dataset
    """
    data_path = os.path.join(dataset_path, 'train' if training else 'test')
    ds = de.ImageFolderDataset(data_path, num_parallel_workers=4,
                              class_indexing=config.class_index)
    resize_height = config.image_height
    resize_width = config.image_width

    normalize_op = C.Normalize(mean=[0.485*255, 0.456*255, 0.406*255], std=[0.229*255,
0.224*255, 0.225*255])
    change_swap_op = C.HWC2CHW()
    type_cast_op = C2.TypeCast(mstype.int32)

    if training:
        crop_decode_resize = C.RandomCropDecodeResize(resize_height, scale=(0.08, 1.0),
ratio=(0.75, 1.333))
        horizontal_flip_op = C.RandomHorizontalFlip(prob=0.5)
        color_adjust = C.RandomColorAdjust(brightness=0.4, contrast=0.4, saturation=0.4)

        train_trans = [crop_decode_resize, horizontal_flip_op, color_adjust, normalize_op,
change_swap_op]
        train_ds = ds.map(input_columns="image", operations=train_trans,
num_parallel_workers=4)
        train_ds = train_ds.map(input_columns="label", operations=type_cast_op,
num_parallel_workers=4)

        train_ds = train_ds.shuffle(buffer_size=buffer_size)
        ds = train_ds.batch(config.batch_size, drop_remainder=True)
    else:
        decode_op = C.Decode()
        resize_op = C.Resize((int(resize_width/0.875), int(resize_width/0.875)))
        center_crop = C.CenterCrop(resize_width)

        eval_trans = [decode_op, resize_op, center_crop, normalize_op, change_swap_op]
        eval_ds = ds.map(input_columns="image", operations=eval_trans,
num_parallel_workers=4)
        eval_ds = eval_ds.map(input_columns="label", operations=type_cast_op,
num_parallel_workers=4)
        ds = eval_ds.batch(config.eval_batch_size, drop_remainder=True)
```

```
return ds
```

展示部分处理后的数据：

```
ds = create_dataset(dataset_path=config.dataset_path, config=config, training=False)
print(ds.get_dataset_size())
data = ds.create_dict_iterator(output_numpy=True)._get_next()
images = data['image']
labels = data['label']

for i in range(1, 5):
    plt.subplot(2, 2, i)
    plt.imshow(np.transpose(images[i], (1,2,0)))
    plt.title('label: %s' % class_en[labels[i]])
    plt.xticks([])
plt.show()
```

展示结果如下图所示：

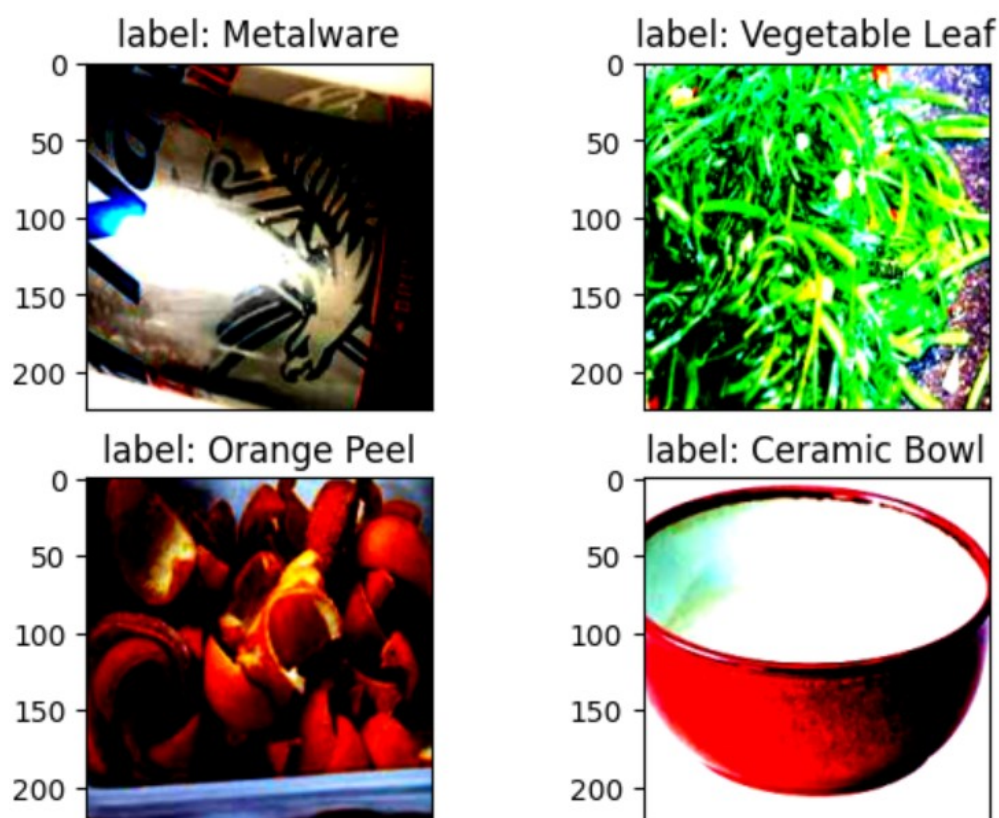


图5-1 测试图片

5.2.5 模型构建

使用 MindSpore 定义 MobileNetV2 网络的各模块时需要继承 `mindspore.nn.Cell`。Cell 是所有神经网络（Conv2d 等）的基类。

神经网络的各层需要预先在 `__init__` 方法中定义，然后通过定义 `construct` 方法来完成神经网络的前向构造。原始模型激活函数为 ReLU6，池化模块采用是全局平均池化层。

```
__all__ = ['MobileNetV2', 'MobileNetV2Backbone', 'MobileNetV2Head', 'mobilenet_v2']

def _make_divisible(v, divisor, min_value=None):
    if min_value is None:
        min_value = divisor
    new_v = max(min_value, int(v + divisor / 2) // divisor * divisor)
    if new_v < 0.9 * v:
        new_v += divisor
    return new_v

class GlobalAvgPooling(nn.Cell):
    """
    Global avg pooling definition.

    Args:

    Returns:
        Tensor, output tensor.

    Examples:
        >>> GlobalAvgPooling()
        """

    def __init__(self):
        super(GlobalAvgPooling, self).__init__()

    def construct(self, x):
        x = P.mean(x, (2, 3))
        return x

class ConvBNReLU(nn.Cell):
    """
    Convolution/Depthwise fused with Batchnorm and ReLU block definition.

    Args:
        in_planes (int): Input channel.
        out_planes (int): Output channel.
        kernel_size (int): Input kernel size.
        stride (int): Stride size for the first convolutional layer. Default: 1.
        groups (int): channel group. Convolution is 1 while Depthwise is input channel. Default: 1.

    Returns:
        Tensor, output tensor.

    Examples:
        >>> ConvBNReLU(16, 256, kernel_size=1, stride=1, groups=1)
        """
```

```

def __init__(self, in_planes, out_planes, kernel_size=3, stride=1, groups=1):
    super(ConvBNReLU, self).__init__()
    padding = (kernel_size - 1) // 2
    in_channels = in_planes
    out_channels = out_planes
    if groups == 1:
        conv = nn.Conv2d(in_channels, out_channels, kernel_size, stride, pad_mode='pad',
padding=padding)
    else:
        out_channels = in_planes
        conv = nn.Conv2d(in_channels, out_channels, kernel_size, stride, pad_mode='pad',
padding=padding, group=in_channels)

    layers = [conv, nn.BatchNorm2d(out_planes), nn.ReLU6()]
    self.features = nn.SequentialCell(layers)

def construct(self, x):
    output = self.features(x)
    return output

class InvertedResidual(nn.Cell):
    """
    Mobilenetv2 residual block definition.

    Args:
        inp (int): Input channel.
        oup (int): Output channel.
        stride (int): Stride size for the first convolutional layer. Default: 1.
        expand_ratio (int): expand ration of input channel

    Returns:
        Tensor, output tensor.

    Examples:
        >>> ResidualBlock(3, 256, 1, 1)
        """

    def __init__(self, inp, oup, stride, expand_ratio):
        super(InvertedResidual, self).__init__()
        assert stride in [1, 2]

        hidden_dim = int(round(inp * expand_ratio))
        self.use_res_connect = stride == 1 and inp == oup

        layers = []
        if expand_ratio != 1:
            layers.append(ConvBNReLU(inp, hidden_dim, kernel_size=1))

```

```

layers.extend([
    ConvBNReLU(hidden_dim, hidden_dim,
                 stride=stride, groups=hidden_dim),
    nn.Conv2d(hidden_dim, oup, kernel_size=1,
               stride=1, has_bias=False),
    nn.BatchNorm2d(oup),
])
self.conv = nn.SequentialCell(layers)
self.cast = P.Cast()

def construct(self, x):
    identity = x
    x = self.conv(x)
    if self.use_res_connect:
        return P.add(identity, x)
    return x

class MobileNetV2Backbone(nn.Cell):
    """
    MobileNetV2 architecture.

    Args:
        class_num (int): number of classes.
        width_mult (int): Channels multiplier for round to 8/16 and others. Default is 1.
        has_dropout (bool): Is dropout used. Default is false
        inverted_residual_setting (list): Inverted residual settings. Default is None
        round_nearest (list): Channel round to . Default is 8
    Returns:
        Tensor, output tensor.

    Examples:
        >>> MobileNetV2(num_classes=1000)
        """

    def __init__(self, width_mult=1., inverted_residual_setting=None, round_nearest=8,
                 input_channel=32, last_channel=1280):
        super(MobileNetV2Backbone, self).__init__()
        block = InvertedResidual
        # setting of inverted residual blocks
        self.cfgs = inverted_residual_setting
        if inverted_residual_setting is None:
            self.cfgs = [
                # t, c, n, s
                [1, 16, 1, 1],
                [6, 24, 2, 2],
                [6, 32, 3, 2],
                [6, 64, 4, 2],
            ]

```

```
[6, 96, 3, 1],
[6, 160, 3, 2],
[6, 320, 1, 1],
]

# building first layer
input_channel = _make_divisible(input_channel * width_mult, round_nearest)
self.out_channels = _make_divisible(last_channel * max(1.0, width_mult), round_nearest)
features = [ConvBNReLU(3, input_channel, stride=2)]
# building inverted residual blocks
for t, c, n, s in self.cfgs:
    output_channel = _make_divisible(c * width_mult, round_nearest)
    for i in range(n):
        stride = s if i == 0 else 1
        features.append(block(input_channel, output_channel, stride, expand_ratio=t))
        input_channel = output_channel
    features.append(ConvBNReLU(input_channel, self.out_channels, kernel_size=1))
self.features = nn.SequentialCell(features)
self._initialize_weights()

def construct(self, x):
    x = self.features(x)
    return x

def _initialize_weights(self):
    """
    Initialize weights.

    Args:

    Returns:
        None.

    Examples:
        >>> _initialize_weights()
        """
    self.init_parameters_data()
    for _, m in self.cells_and_names():
        if isinstance(m, nn.Conv2d):
            n = m.kernel_size[0] * m.kernel_size[1] * m.out_channels
            m.weight.set_data(Tensor(np.random.normal(0, np.sqrt(2. / n),
                                                    m.weight.data.shape).astype("float32")))
            if m.bias is not None:
                m.bias.set_data(
                    Tensor(np.zeros(m.bias.data.shape, dtype="float32")))
        elif isinstance(m, nn.BatchNorm2d):
            m.gamma.set_data(
```

```
        Tensor(np.ones(m.gamma.data.shape, dtype="float32"))
        m.beta.set_data(
            Tensor(np.zeros(m.beta.data.shape, dtype="float32")))

    @property
    def get_features(self):
        return self.features

class MobileNetV2Head(nn.Cell):
    """
    MobileNetV2 architecture.

    Args:
        class_num (int): Number of classes. Default is 1000.
        has_dropout (bool): Is dropout used. Default is false
    Returns:
        Tensor, output tensor.

    Examples:
        >>> MobileNetV2(num_classes=1000)
        """

    def __init__(self, input_channel=1280, num_classes=1000, has_dropout=False,
                 activation="None"):
        super(MobileNetV2Head, self).__init__()
        # mobilenet head
        head = ([GlobalAvgPooling(), nn.Dense(input_channel, num_classes, has_bias=True)] if
        not has_dropout else
                [GlobalAvgPooling(), nn.Dropout(0.2), nn.Dense(input_channel, num_classes,
                has_bias=True)])
        self.head = nn.SequentialCell(head)
        self.need_activation = True
        if activation == "Sigmoid":
            self.activation = nn.Sigmoid()
        elif activation == "Softmax":
            self.activation = nn.Softmax()
        else:
            self.need_activation = False
        self._initialize_weights()

    def construct(self, x):
        x = self.head(x)
        if self.need_activation:
            x = self.activation(x)
        return x

    def _initialize_weights(self):
        """
```



```
Initialize weights.

Args:

Returns:
    None.

Examples:
    >>> _initialize_weights()
    """
    self.init_parameters_data()
    for _, m in self.cells_and_names():
        if isinstance(m, nn.Dense):
            m.weight.set_data(Tensor(np.random.normal(
                0, 0.01, m.weight.data.shape).astype("float32")))
            if m.bias is not None:
                m.bias.set_data(
                    Tensor(np.zeros(m.bias.data.shape, dtype="float32")))
    @property
    def get_head(self):
        return self.head

class MobileNetV2(nn.Cell):
    """
    MobileNetV2 architecture.

    Args:
        class_num (int): number of classes.
        width_mult (int): Channels multiplier for round to 8/16 and others. Default is 1.
        has_dropout (bool): Is dropout used. Default is false
        inverted_residual_setting (list): Inverted residual settings. Default is None
        round_nearest (list): Channel round to . Default is 8
    Returns:
        Tensor, output tensor.

    Examples:
        >>> MobileNetV2(backbone, head)
        """

    def __init__(self, num_classes=1000, width_mult=1., has_dropout=False,
        inverted_residual_setting=None, \
        round_nearest=8, input_channel=32, last_channel=1280):
        super(MobileNetV2, self).__init__()
        self.backbone = MobileNetV2Backbone(width_mult=width_mult, \
            inverted_residual_setting=inverted_residual_setting, \
            round_nearest=round_nearest, input_channel=input_channel,
            last_channel=last_channel).get_features
```

```

        self.head = MobileNetV2Head(input_channel=self.backbone.out_channel,
num_classes=num_classes, \
        has_dropout=has_dropout).get_head

def construct(self, x):
    x = self.backbone(x)
    x = self.head(x)
    return x

class MobileNetV2Combine(nn.Cell):
    """
    MobileNetV2Combine architecture.

    Args:
        backbone (Cell): the features extract layers.
        head (Cell): the fully connected layers.
    Returns:
        Tensor, output tensor.

    Examples:
        >>> MobileNetV2(num_classes=1000)
    """

    def __init__(self, backbone, head):
        super(MobileNetV2Combine, self).__init__(auto_prefix=False)
        self.backbone = backbone
        self.head = head

    def construct(self, x):
        x = self.backbone(x)
        x = self.head(x)
        return x

def mobilenet_v2(backbone, head):
    return MobileNetV2Combine(backbone, head)

```

5.2.6 模型训练与验证

步骤 1 动态学习率

一般情况下，模型训练时采用静态学习率，如 0.01。随着训练步数的增加，模型逐渐趋于收敛，对权重参数的更新幅度应该逐渐降低，以减小模型训练后期的抖动。所以模型训练时可以采用动态下降的学习率，这里使用 cosine decay 下降策略。

```

def cosine_decay(total_steps, lr_init=0.0, lr_end=0.0, lr_max=0.1, warmup_steps=0):
    """
    Applies cosine decay to generate learning rate array.

    Args:

```

```

total_steps(int): all steps in training.
lr_init(float): init learning rate.
lr_end(float): end learning rate
lr_max(float): max learning rate.
warmup_steps(int): all steps in warmup epochs.

Returns:
    list, learning rate array.
"""
lr_init, lr_end, lr_max = float(lr_init), float(lr_end), float(lr_max)
decay_steps = total_steps - warmup_steps
lr_all_steps = []
inc_per_step = (lr_max - lr_init) / warmup_steps if warmup_steps else 0
for i in range(total_steps):
    if i < warmup_steps:
        lr = lr_init + inc_per_step * (i + 1)
    else:
        cosine_decay = 0.5 * (1 + math.cos(math.pi * (i - warmup_steps) / decay_steps))
        lr = (lr_max - lr_end) * cosine_decay + lr_end
    lr_all_steps.append(lr)

return lr_all_steps

```

步骤 2 模型训练

在模型训练过程中，可以添加检查点（Checkpoint）保存模型参数，适用于推理或再训练场景。

- 推理场景：
 - 1) 模型训练完毕后保存模型的参数，用于推理或预测操作。
 - 2) 训练过程中，通过实时验证精度，把精度最高的模型参数保存下来，用于预测操作。
- 再训练场景：
 - 1) 进行长时间训练任务时，保存训练过程中的 Checkpoint 文件，防止任务异常退出。
 - 2) 微调（Fine-tuning）场景，即训练一个模型并保存参数，基于该模型，面向第二个类似任务进行模型训练。

这里加载在 ImageNet 数据上做预训练的 MobileNetv2 模型

（mobilenetv2_ascend.ckpt）进行微调，只训练最后的 FC 层，并在训练过程中保存 Checkpoint。

```

def switch_precision(net, data_type):
    if ms.get_context('device_target') == "Ascend":
        net.to_float(data_type)
        for _, cell in net.cells_and_names():
            if isinstance(cell, nn.Dense):
                cell.to_float(ms.float32)

```

在进行正式的训练之前，定义训练函数，读取数据并对模型进行实例化，定义优化器和损失函数。

首先简单介绍损失函数及优化器的概念：

- 损失函数：又叫目标函数，用于衡量预测值与实际值差异的程度。深度学习通过不停地迭代来缩小损失函数的值。定义一个好的损失函数，可以有效提高模型的性能。
- 优化器：用于最小化损失函数，从而在训练过程中改进模型。

定义了损失函数后，可以得到损失函数关于权重的梯度。梯度用于指示优化器优化权重的方向，以提高模型性能。

在训练 MobileNetV2 之前对 MobileNetV2Backbone 层的参数进行了固定，使其在训练过程中对该模块的权重参数不进行更新；只对 MobileNetV2Head 模块的参数进行更新。

MindSpore 支持的损失函数有 SoftmaxCrossEntropyWithLogits、L1Loss、MSELoss 等。这里使用 SoftmaxCrossEntropyWithLogits 损失函数。

训练测试过程中会打印 loss 值，loss 值会波动，但总体来说 loss 值会逐步减小，精度逐步提高。每个人运行的 loss 值有一定随机性，不一定完全相同。

每打印一个 epoch 后模型都会在测试集上的计算测试精度，从打印的精度值分析 MobileNetV2 模型的预测能力在不断提升。

```
from mindspore.amp import FixedLossScaleManager
LOSS_SCALE = 1024

train_dataset = create_dataset(dataset_path=config.dataset_path, config=config)
eval_dataset = create_dataset(dataset_path=config.dataset_path, config=config)
step_size = train_dataset.get_dataset_size()

backbone = MobileNetV2Backbone() #last_channel=config.backbone_out_channels
# Freeze parameters of backbone. You can comment these two lines.
for param in backbone.get_parameters():
    param.requires_grad = False
# load parameters from pretrained model
load_checkpoint(config.pretrained_ckpt, backbone)

head = MobileNetV2Head(input_channel=backbone.out_channels,
num_classes=config.num_classes)
network = mobilenet_v2(backbone, head)

# define loss, optimizer, and model
loss = nn.SoftmaxCrossEntropyWithLogits(sparse=True, reduction='mean')
loss_scale = FixedLossScaleManager(LOSS_SCALE, drop_overflow_update=False)
lrs = cosine_decay(config.epochs * step_size, lr_max=config.lr_max)
opt = nn.Momentum(network.trainable_params(), lrs, config.momentum, config.weight_decay,
loss_scale=LOSS_SCALE)

# 定义用于训练的 train_loop 函数。
def train_loop(model, dataset, loss_fn, optimizer):
    # 定义正向计算函数
    def forward_fn(data, label):
        logits = model(data)
        loss = loss_fn(logits, label)
```

```

return loss

# 定义微分函数，使用 mindspore.value_and_grad 获得微分函数 grad_fn,输出 loss 和梯度。
# 由于是对模型参数求导,grad_position 配置为 None，传入可训练参数。
grad_fn = ms.value_and_grad(forward_fn, None, optimizer.parameters)

# 定义 one-step training 函数
def train_step(data, label):
    loss, grads = grad_fn(data, label)
    optimizer(grads)
    return loss

size = dataset.get_dataset_size()
model.set_train()
for batch, (data, label) in enumerate(dataset.create_tuple_iterator()):
    loss = train_step(data, label)

    if batch % 10 == 0:
        loss, current = loss.asnumpy(), batch
        print(f"loss: {loss:>7f} [{current:>3d}/{size:>3d}]")

# 定义用于测试的 test_loop 函数。
def test_loop(model, dataset, loss_fn):
    num_batches = dataset.get_dataset_size()
    model.set_train(False)
    total, test_loss, correct = 0, 0, 0
    for data, label in dataset.create_tuple_iterator():
        pred = model(data)
        total += len(data)
        test_loss += loss_fn(pred, label).asnumpy()
        correct += (pred.argmax(1) == label).asnumpy().sum()
    test_loss /= num_batches
    correct /= total
    print(f"Test: \n Accuracy: {(100*correct)>0.1f}%, Avg loss: {test_loss:>8f} \n")
print("===== Starting Training =====")
epochs = 10
for t in range(epochs):
    print(f"Epoch {t+1}\n-----")
    train_loop(network, train_dataset, loss, opt)
    ms.save_checkpoint(network, "./ckpt/save_mobilenetV2_model.ckpt")
    test_loop(network, eval_dataset, loss)
print("Done!")

```

训练输出如下，模型保存在 mobilenetv2_garbage/ckpt/
save_mobilenetV2_model.ckpt：

```

===== Starting Training =====
Epoch 1
-----

```

```
loss: 3.365921 [ 0/162]
loss: 3.278985 [ 10/162]
.....
Epoch 10
-----
.....
loss: 2.831076 [140/162]
loss: 2.920157 [150/162]
loss: 2.913762 [160/162]
Test:
Accuracy: 51.7%, Avg loss: 2.881466

Done!
```

步骤 3 模型推理

加载 `save_mobilenetV2_model.ckpt` 对 `data_en/test/Cardboard` 内的图片进行推理，使用 `load_checkpoint` 接口加载数据时，需要把数据传入给原始网络，而不能传递给带有优化器和损失函数的训练网络。

```
CKPT="save_mobilenetV2_model.ckpt"
def image_process(image):
    """Precess one image per time.

    Args:
        image: shape (H, W, C)
    """
    mean=[0.485*255, 0.456*255, 0.406*255]
    std=[0.229*255, 0.224*255, 0.225*255]
    image = (np.array(image) - mean) / std
    image = image.transpose((2,0,1))
    img_tensor = Tensor(np.array([image], np.float32))
    return img_tensor

def infer_one(network, image_path):
    image = Image.open(image_path).resize((config.image_height, config.image_width))
    logits = network(image_process(image))
    pred = np.argmax(logits.asnumpy(), axis=1)[0]
    print(image_path, class_en[pred])

def infer():
    backbone = MobileNetV2Backbone(last_channel=config.backbone_out_channels)
    head = MobileNetV2Head(input_channel=backbone.out_channels,
        num_classes=config.num_classes)
    network = mobilenet_v2(backbone, head)
    load_checkpoint(os.path.join(config.save_ckpt_path, CKPT), network)
    for i in range(91, 100):
        infer_one(network, f'data_en/test/Cardboard/000{i}.jpg')
    infer()
```

输出：

```
data_en/test/Cardboard/00091.jpg Plastic Bottle
data_en/test/Cardboard/00092.jpg Disposable Chopsticks
data_en/test/Cardboard/00093.jpg Cardboard
data_en/test/Cardboard/00094.jpg Cardboard
data_en/test/Cardboard/00095.jpg Cardboard
data_en/test/Cardboard/00096.jpg Glassware
data_en/test/Cardboard/00097.jpg Cardboard
data_en/test/Cardboard/00098.jpg Cardboard
data_en/test/Cardboard/00099.jpg Cardboard
```

步骤 4 导出 AIR 模型文件

导出 AIR 模型文件，用于后续在 Atlas 200I DK A2 上的模型转换与推理。

```
backbone = MobileNetV2Backbone(last_channel=config.backbone_out_channels)
head = MobileNetV2Head(input_channel=backbone.out_channels,
num_classes=config.num_classes)
network = mobilenet_v2(backbone, head)
load_checkpoint(os.path.join(config.save_ckpt_path, CKPT), network)

input = np.random.uniform(0.0, 1.0, size=[1, 3, 224, 224]).astype(np.float32)
#export(network, Tensor(input), file_name='mobilenetv2.air', file_format='AIR')
# export(network, Tensor(input), file_name='mobilenetv2.pb', file_format='GEIR')
export(network, Tensor(input), file_name='mobilenetv2.air', file_format='AIR')
```

如使用华为云上环境，记得将生成的 mobilenetv2.air 文件下载到本机：

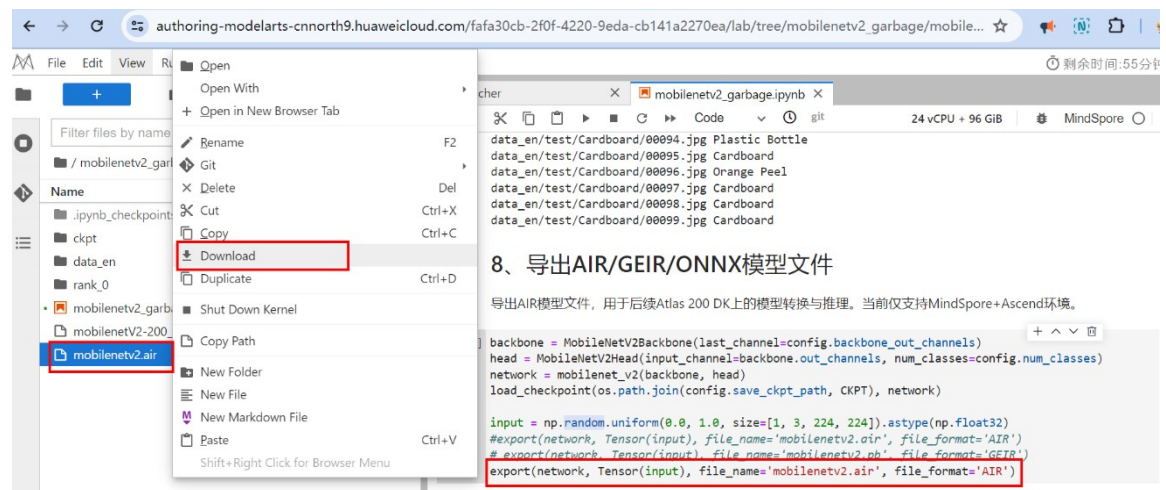


图5-1 下载 mobilenetv2.air 模型

步骤 5 如使用华为云环境，请及时关闭云环境

实验完成之后，请及时关闭华为云 ModelArts 的 Notebook 开发环境，避免资源浪费。

关闭方式：登录[华为云 ModelArts 控制台](#)，在“操作”栏选择“停止”或“更多—>删除”操作。

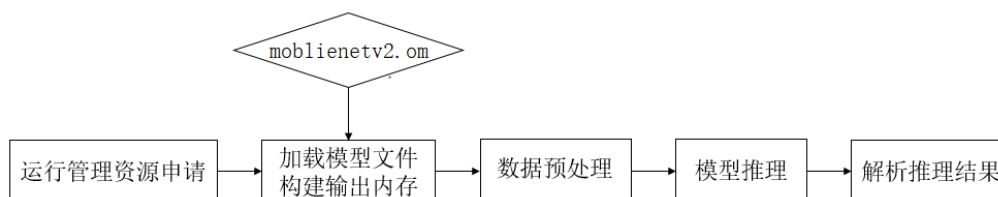
如下图所示：



图5-1 及时关闭/删除云环境

5.3 Atlas 200I DK A2 端侧部署

本实验使用昇腾 CANN 在 Atlas 200I DK A2 上开发垃圾分类推理应用，首先使用 ATC 工具对 mobilenetv2.air 模型进行离线模型转换，接下来使用 AscendCL 开发推理代码，对图像中的物体进行分类推理，最后使用 OpenCV 写到本地文件中。输入样例是待推理的 jpg 图片，输出样例是推理后的 jpg 图片。



1. 运行管理资源申请：用于初始化系统内部资源，固定的调用流程。
2. 加载模型文件并构建输出内存：从文件加载离线模型 mobilenetv2.om 数据，需要由用户自行管理模型运行的内存，根据内存中加载的模型获取模型的基本信息包含模型输入、输出数据的数据 buffer 大小；由模型的基本信息构建模型输出内存，为接下来的模型推理做好准备。
3. 数据预处理：对读入的图像数据进行预处理，然后构建模型的输入数据。
4. 模型推理：根据构建好的模型输入数据进行模型推理。
5. 解析推理结果：根据模型输出，解析模型的推理结果。使用 OpenCV 将垃圾分类的结果写进图片。

5.3.1 环境准备

本实验是在新一代开发者套件 Atlas 200I DK A2 上执行，因此需要部署 Atlas 200I DK A2 的实验环境，请参考《手把手教你搭建 Atlas 200I DK A2 开发环境》实验手册。

5.3.2 下载项目

步骤 1 下载项目代码

```
git clone https://gitee.com/ascend/samples.git
```


本实验项目代码位于 root/samples/python/contrib/garbage_picture/model
项目目录如下：

```
root/samples/python/contrib/garbage_picture/model
├─data # 需创建
│   # 测试图片
├─model
│   # 推理模型
├─src
│   classify_test.py # 推理代码
```

步骤 2 在 model 目录上传或下载 mobilenetv2.air 模型

(1) 将“[5.2.5 模型训练与验证 garbage.ipynb](#) 步骤 6 导出 AIR 模型文件”保存的 mobilenetv2.air 模型上传至 root/samples/python/contrib/garbage_picture/model 目录。

或者 (2) 在 model 目录下载基于 MindSpore 的 mobilenetv2.air 模型：

```
cd ${HOME}/samples/python/contrib/garbage_picture/model
wget https://obs-9be7.obs.cn-east-2.myhuaweicloud.com:443/003_Atc_Models/AE/ATC%20Model/garbage/mobilenetv2.air
```

步骤 3 创建 data 文件夹

```
mkdir $HOME/samples/python/contrib/garbage_picture/data
```

步骤 4 在 data 目录下载测试图片

```
cd $HOME/samples/python/contrib/garbage_picture/data
wget https://obs-9be7.obs.cn-east-2.myhuaweicloud.com/models/garbage_picture/newspaper.jpg
wget https://obs-9be7.obs.cn-east-2.myhuaweicloud.com/models/garbage_picture/bottle.jpg
wget https://obs-9be7.obs.cn-east-2.myhuaweicloud.com/models/garbage_picture/dirtycloth.jpg
```



图5-1 测试图片样例

5.3.3 离线模型转换

步骤 1 使用 ATC 工具进行离线模型转换

使用 MindSpore 训练的 MobileNetV2 模型在转换时需要准备模型文件 mobilenetv2.air 和 AIPP 配置文件 insert_op_yuv.cfg。

注意：这里的 soc_version 是使用 npu-smi info 查询到的 Ascend310B4。

```
cd ${HOME}/samples/python/contrib/garbage_picture/model
wget https://obs-9be7.obs.cn-east-2.myhuaweicloud.com/models/garbage_picture/
insert_op_yuv.cfg
atc --model=./mobilenetv2.air --framework=1 --output=garbage_yuv --
soc_version=Ascend310B4 --insert_op_conf=./insert_op_yuv.cfg --
input_shape="data:1,3,224,224" --input_format=NCHW
```

输出“ATC Run Success”代表模型转换成功：

```
(base) root@davinci-mini:~/samples/python/contrib/garbage_picture/model# atc --model=./mobilenetv2.air --framework=
1 --output=garbage_yuv --soc_version=Ascend310B4 --insert_op_conf=./insert_op_yuv.cfg --input_shape="data:1,3,224,2
24" --input_format=NCHW
ATC start working now, please wait for a moment.
...
ATC run success, welcome to the next use.
```

图5-1 模型转换成功

5.3.4 执行模型推理

步骤 1 执行模型推理

classify_test.py 为模型推理代码，对 data 文件夹内的图片进行推理。

```
cd ${HOME}/samples/python/contrib/garbage_picture/src
python3 classify_test.py ../data/
```

```
(base) root@davinci-mini:~/samples/python/contrib/garbage_picture/src# cd ${HOME}/samples/python/contrib/garbage_picture/src
(base) root@davinci-mini:~/samples/python/contrib/garbage_picture/src# python3 classify_test.py ../data/
[INFO] init resource stage:
[INFO] Init resource success
[INFO] Init model resource start...
[INFO] AclLiteModel create model output dataset:
[INFO] malloc output 0, size 104
[INFO] Create model output dataset success
[INFO] Init model resource success
decode jpeg end
resize yuv end
pre process end
post process
process ../data/dirtycloth.jpg end
decode jpeg end
resize yuv end
pre process end
post process
process ../data/bottle.jpg end
decode jpeg end
resize yuv end
pre process end
post process
process ../data/newspaper.jpg end
[INFO] acl resource release all resource
[INFO] AclLiteModel release source success
[INFO] dvpp resource release success
[INFO] acl resource release stream
[INFO] acl resource release context
[INFO] Reset acl device 0
[INFO] Release acl resource success
```

图5-1 模型推理结果

步骤 2 查看推理结果

推理完成后在 out 文件夹内得到推理结果图片：



图5-1 推理结果图片

5.4 实验小结

本实验是基于 Ascend910 和 Ascend310B 算力平台，实现垃圾分类模型训练+推理部署的全流程实验。在 Ascend910 算力平台使用迁移学习训练得到 MobileNetv2 的垃圾分类模型，将其部署在 Atlas 200I DK A2 上，使用 Ascend310B 算力平台实现端侧推理。通过本实验

使学员掌握 MindSpore 的算法开发，昇腾 CANN 的推理应用开发，Ascend910 和 Ascend310B 算力平台的使用。